

Laporan Tugas Besar 4

TBFO (IF2224)



Disusun Oleh

Kelompok GooBurs:

Eliana Natalie Widjojo (13524116)

Muhammad Rafi Akbar (13524125)

Varistha Devi (13524135)

Ahmad Rinofaros Muchtar (13524138)

SEKOLAH TEKNIK ELEKTRO DAN INFORMATIKA

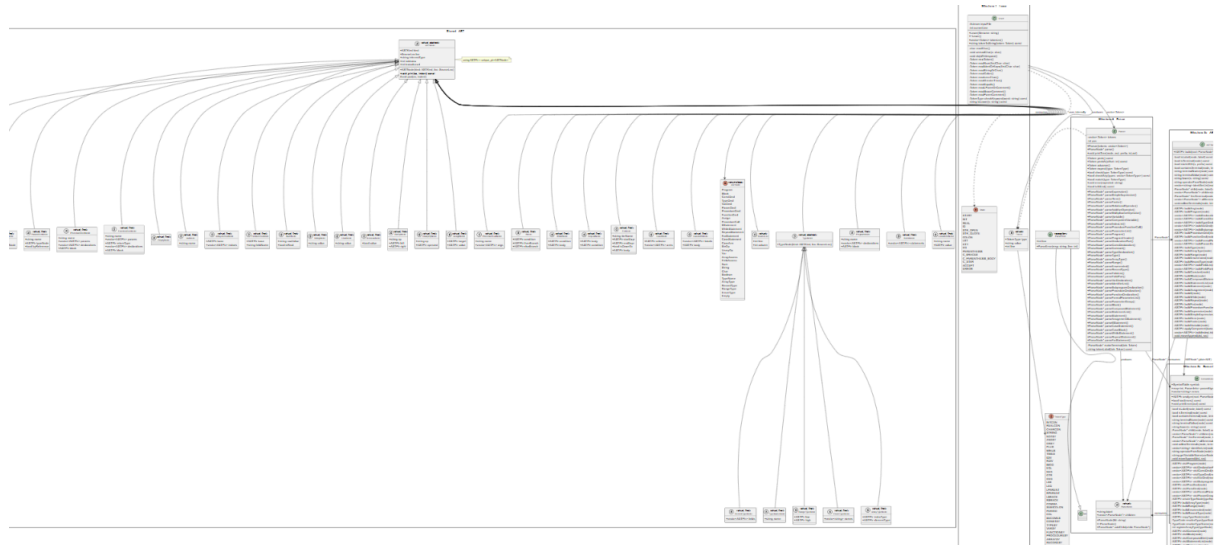
INSTITUT TEKNOLOGI BANDUNG

BANDUNG 2025

Daftar Isi

Daftar Isi	2
Bab 1. Teori Singkat	3
1. Abstract Syntax Tree (AST)	3
2. Semantic Analysis	4
a. Symbol Table	5
b. Type Checking	6
c. Scope Resolution	7
d. Semantic Errors	7
e. Attribute Grammar dan Syntax Directed Translation	8
f. Decorated AST	8
3. Enumerated Type	9
Bab 2. Perancangan dan Implementasi	12
1. ast_builder	12
a. Fungsi Navigasi Parse Tree	12
b. Pembangunan Struktur Program	12
c. Pembangunan Tipe	12
d. Pembangunan Pernyataan dan Ekspresi	13
2. symbol_table	13
a. Struktur	13
b. Inisialisasi Predetermined	13
c. Scope management	14
3. semantic_analyzer	14
a. Entry Point dan Alur Umum	14
b. Fungsi Visit	14
c. Type System	16
d. Semantic Checking dan Error Handling	17
e. Penanganan Enumerated Type	17
f. Dekorasi Node AST	18
Bab 3. Pengujian	19
1. Kasus 1	19
2. Kasus 2	25
3. Kasus 3	37
4. Kasus 4	47
5. Kasus 5	56
6. Kasus 6	63
Bab 4. Kesimpulan dan Saran	70
Hasil Uji	70
Saran	70
Lampiran	70
Referensi	71

Class Diagram



Full picture:  ClassDiagramM14.svg

Bab 1. Teori Singkat

1. Intermediate Representation

Intermediate Representation (IR) adalah bentuk perantara dari sebuah program yang terletak di antara AST (Abstract Syntax Tree) yang dihasilkan oleh front-end compiler dan *machine code*. IR dirancang agar:

- Mudah dihasilkan dari AST
- Mudah dieksekusi oleh interpreter atau mesin virtual
- Bersifat linear, tidak hierarkis seperti AST
- Independen terhadap bahasa sumber ataupun arsitektur mesin target

Dalam compiler modern, IR berperan sebagai jembatan/*bridge* yang memisahkan dua fase utama

Fase	Input	Output
Front-end	Source Code	Decorated AST
IR Generator	Decorated AST	Intermediate Code
Back-end/Interpreter	Intermediate Code	Hasil Eksekusi

Keberadaan IR penting untuk alasan berikut:

- a. AST bersifat Hierarkis, Interpreter bersifat Linear
- b. Memisahkan Front-end dan Back-end
- c. Memungkinkan Optimasi
- d. Menyederhanakan Eksekusi oleh Interpreter

2. Three Address Code (TAC)

Three Address Code (TAC) adalah bentuk intermediate representation (IR) di mana setiap instruksi mengandung paling banyak tiga alamat (address), yang biasanya terdiri dari:

- Dua operand sumber (source operands)
- Satu hasil tujuan (destination result)

Dengan format dasar:

result = operand1 operator operand2

atau dapat dalam variasi lain sebagai:

operator result, operand1, operand2 result = operand1
--

Bentuk ini dinamakan “Three Address” karena setiap instruksi dapat direferensikan melalui tiga alamat: dua untuk operand dan satu untuk hasil.

Jenis-Jenis Instruksi TAC:

Jenis Instruksi	Format	Contoh
Assignment	$x = y$	$i = 10$
Binary operation	$x = y \text{ op } z$	$t1 = a + b$
Unary operation	$x = \text{op } y$	$t1 = -x$
Copy	$x = y$	$t2 = t1$
Unconditional jump	goto L	goto L_start
Conditional jump	if x rel op y goto L	if $i < 5$ goto L_body
Procedure call	call p, args	call add, 3, 4
Return	return x	return t1
Array access	$x = y[z]$	$t1 = \text{arr}[i]$
Array assignment	$x[y] = z$	$\text{arr}[i] = t1$

Dalam instruksi TAC, penting untuk menggunakan Temporary Variable, variabel sementara yang dibuat secara otomatis oleh compiler/IR generator untuk menyimpan hasil antara (intermediate results) dari ekspresi kompleks. Variabel ini tidak dideklarasikan oleh programmer, namun di-generate oleh sistem dengan nama seperti t1, t2, t3, dst.

Ekspresi	TAC yang Dihasilkan
$b * c$	$t1 = b * c$
$a + t1$	$t2 = a + t1$
Hasil: $x = t2$	

Tanpa temporary variable, ekspresi bersarang tidak dapat dinyatakan dalam format linear TAC yang hanya memiliki satu operator per instruksi.

3. Intermediate Code Instructions

Intermediate Code Instructions adalah kumpulan instruksi yang membentuk sebuah IR. Dalam konteks interpreter stack-based (seperti yang umum dibangun secara handmade), instruksi-instruksi ini direpresentasikan sebagai opcode — bilangan bulat yang merepresentasikan operasi tertentu.

Setiap instruksi umumnya terdiri dari:

- Opcode: jenis operasi (misal: ADD, LOAD, JUMP)
- Operand (opsional): nilai atau alamat yang digunakan instruksi tersebut

Contoh representasi instruksi dalam bentuk struct:

```
typedef struct {
    int opcode;
    int operand;
} Instruction;
```

Instruksi-instruksi ini disimpan dalam sebuah array (instruction array / code store) dan dieksekusi secara sekuensial oleh interpreter menggunakan sebuah program counter (PC).

4. Operasi-operasi OPR

OPR (Operation) adalah jenis instruksi khusus dalam banyak implementasi interpreter handmade (terinspirasi dari PL/0 dan Wirth's compiler) yang merepresentasikan operasi aritmatika dan logika. Alih-alih menggunakan opcode terpisah untuk setiap operasi, satu instruksi OPR digunakan dengan operand yang menentukan jenis operasinya.

Kode OPR	Operasi	Keterangan
OPR 0	RETURN	Kembali dari prosedur
OPR 1	NEG	Negasi (unary minus)
OPR 2	ADD	Penjumlahan
OPR 3	SUB	Pengurangan

OPR 4	MUL	Perkalian
OPR 5	DIV	Pembagian
OPR 6	ODD	Cek apakah ganjil
OPR 7	EQL	Sama dengan (==)
OPR 8	NEQ	Tidak sama (!=)
OPR 9	LSS	Kurang dari (<)
OPR 10	GEQ	Lebih dari atau sama (>=)
OPR 11	GTR	Lebih dari (>)
OPR 12	LEQ	Kurang dari atau sama (<=)

Saat interpreter menemui instruksi OPR, ia mengambil satu atau dua nilai dari stack, melakukan operasi sesuai kode, lalu mendorong hasilnya kembali ke stack.

5. Stack Machine Architecture

Stack Machine (mesin berbasis tumpukan) adalah arsitektur eksekusi di mana semua operasi dilakukan menggunakan sebuah stack (tumpukan). Tidak ada register eksplisit; operand diambil dari stack dan hasil dikembalikan ke stack.

Komponen utama sebuah stack machine:

Komponen	Fungsi
Stack	Menyimpan nilai sementara, argumen, dan variabel lokal
Program Counter (PC)	Menunjuk instruksi berikutnya yang akan dieksekusi
Stack Pointer (SP)	Menunjuk ke top of stack
Base Pointer (BP)	Menunjuk ke base frame prosedur aktif
Code Store	Array berisi seluruh instruksi program

Contoh eksekusi ekspresi $3 + 4$ pada stack machine:

PUSH 3	→ stack: [3]
PUSH 4	→ stack: [3, 4]
OPR ADD	→ pop 4 dan 3, push 7 → stack: [7]

6. Interpreter dan Virtual Machine

Interpreter adalah program yang membaca dan mengeksekusi intermediate code secara langsung, tanpa menghasilkan machine code native. Sedangkan Virtual Machine (VM) adalah interpreter yang mensimulasikan sebuah "mesin abstrak" lengkap dengan stack, memori, dan instruksi set sendiri.

Aspek	Interpreter	Compiler
Eksekusi	Langsung dari IR	Dari machine code
Kecepatan	Lebih lambat	Lebih cepat
Portabilitas	Tinggi	Rendah (target-specific)
Debugging	Lebih mudah	Lebih sulit

7. Activation Record (Stack Frame)

Activation Record/Stack Frame adalah blok memori yang dialokasikan di stack setiap kali sebuah prosedur/fungsi dipanggil. Blok ini menyimpan semua informasi yang dibutuhkan selama eksekusi prosedur tersebut.

Struktur umum sebuah activation record:

	<- Top of Stack
Variabel Lokal	
Temporary Values	
Return Value	
Dynamic Link	-> BP pemanggil
Static Link	-> BP lingkup leksikal induk
Return Address	-> PC setelah pemanggilan
	<- BP (Base of Frame)

Saat prosedur dipanggil, interpreter:

1. Menyimpan BP lama sebagai Dynamic Link
2. Mengatur BP baru ke posisi frame baru
3. Mengalokasikan ruang untuk variabel lokal

Saat prosedur selesai, frame dihancurkan dan BP dikembalikan ke nilai sebelumnya.

8. Lexical Scope dan Static Link

Lexical Scope (atau Static Scope) adalah aturan di mana variabel diakses berdasarkan struktur teks program (nesting level), bukan berdasarkan urutan pemanggilan saat runtime. Ini berarti sebuah prosedur dapat mengakses variabel dari prosedur yang secara leksikal membungkusnya (enclosing scope).

Static Link adalah pointer di dalam activation record yang menunjuk ke activation record dari prosedur induk secara leksikal (bukan pemanggil secara dinamis). Static link digunakan untuk menelusuri variabel dari scope yang lebih luar.

```
procedure A:      -- level 1
  var x
  procedure B:    -- level 2
    var y
    procedure C:  -- level 3
      x := 10     -- mengakses variabel milik A (2 level ke atas)
```

Untuk mengakses x dari dalam C, interpreter menelusuri static link sebanyak selisih level leksikal (2 kali):

contoh:

```
C.static_link → B.activation_record
B.static_link → A.activation_record → ambil x
```

Proses penelusuran ini disebut display traversal atau static chain traversal, dengan jumlah lompatan sebesar $\text{current_level} - \text{variable_level}$.

9. Translasi Control Flow

Control flow mencakup struktur seperti if-then-else, while, dan for yang mengubah alur eksekusi program. Dalam IR/TAC, control flow diterjemahkan menggunakan instruksi jump (lompatan) ke label atau alamat instruksi tertentu.

Translasi if-then-else:

```
if (kondisi) then A else B
```

Menghasilkan TAC:

```
if kondisi == false goto L_else
  [instruksi A]
  goto L_end
L_else:
  [instruksi B]
L_end:
```

Translasi while:


```
while (kondisi) do body
```

Menghasilkan TAC:

```
L_start:  
  if kondisi == false goto L_end  
  [instruksi body]  
  goto L_start  
L_end:
```

Pada implementasi handmade, label direalisasikan sebagai indeks numerik dalam array instruksi, dan instruksi jump langsung memodifikasi nilai Program Counter (PC).

10. Runtime Error Handling

Runtime Error adalah kesalahan yang terjadi saat program dieksekusi (bukan saat kompilasi), seperti pembagian dengan nol, akses array di luar batas, atau stack overflow. Dalam interpreter handmade, penanganan error ini dilakukan langsung di dalam loop eksekusi. Jenis-jenis runtime error yang umum ditangani:

Jenis Error	Kondisi	Penanganan
Division by zero	divisor == 0 sebelum OPR DIV	Hentikan eksekusi, tampilkan pesan
Stack overflow	SP melebihi batas maksimum stack	Hentikan eksekusi
Stack underflow	Pop pada stack kosong	Hentikan eksekusi
Invalid jump	PC lompat ke alamat di luar code store	Hentikan eksekusi
Undefined variable	Akses ke alamat memori tidak valid	Hentikan eksekusi

Pendekatan paling umum dalam implementasi sederhana adalah fail-fast: begitu error terdeteksi, interpreter langsung menghentikan eksekusi dan mencetak pesan diagnostik yang informatif, termasuk nomor instruksi (PC) saat error terjadi untuk memudahkan debugging.

Bab 2. Perancangan dan Implementasi

1. Gambaran Umum Sistem

Pada Milestone 4, compiler Arion dikembangkan agar mampu menerjemahkan Decorated Abstract Syntax Tree (AST) menjadi intermediate code dan menjalankan instruksi tersebut

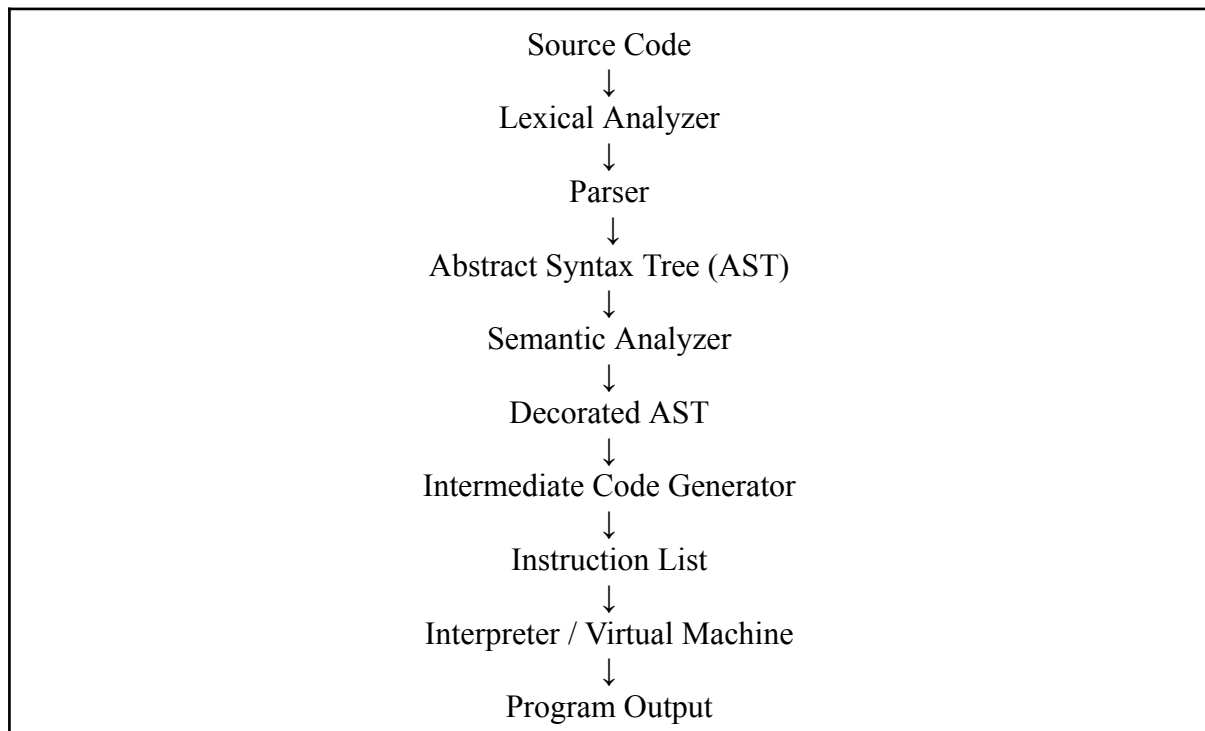
menggunakan interpreter berbasis stack machine. Berdasarkan spesifikasi milestone, fokus utama implementasi meliputi:

- intermediate code generation,
- translasi control flow,
- stack-based execution,
- procedure/function call,
- dan runtime execution.

Implementasi pada milestone ini melanjutkan hasil milestone sebelumnya, dimana compiler telah mampu melakukan:

- lexical analysis,
- syntax analysis,
- semantic analysis,
- serta menghasilkan Decorated AST.

Secara umum, alur kerja compiler Arion pada Milestone 4 dapat digambarkan sebagai berikut:



Pada tahap code generation, compiler melakukan traversal terhadap Decorated AST dan menghasilkan instruksi intermediate code berbasis stack machine. Instruksi tersebut kemudian dieksekusi secara sequential oleh interpreter menggunakan virtual memory berbasis stack.

Pendekatan stack machine dipilih karena lebih sederhana dibandingkan register-based machine dan mempermudah translasi expression maupun control flow menjadi instruksi executable.

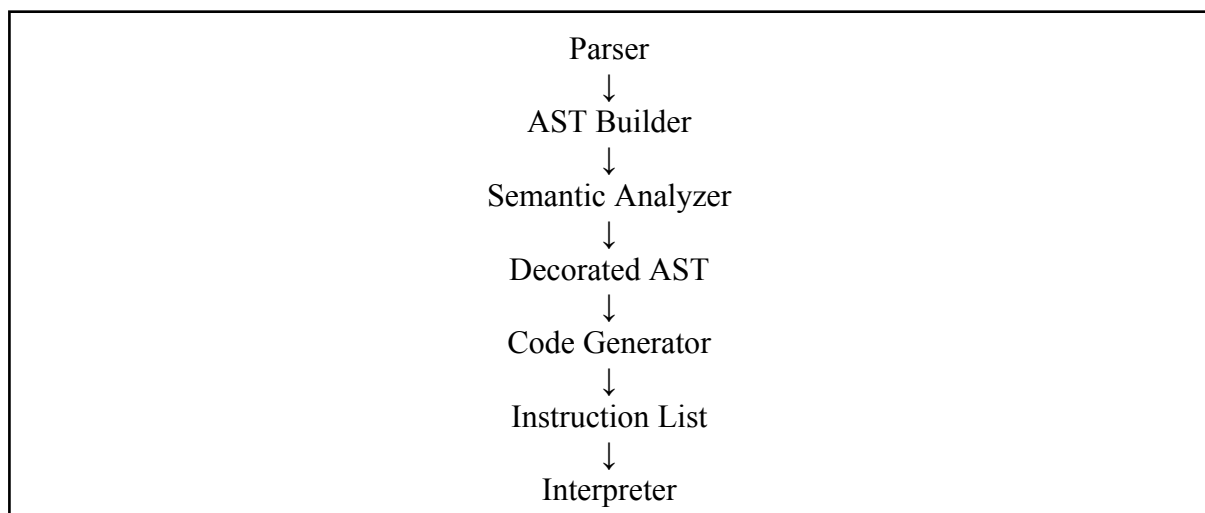
2. Arsitektur Program

Compiler Arion dibangun menggunakan pendekatan modular agar setiap komponen memiliki tanggung jawab yang terpisah dengan jelas. Pendekatan ini mempermudah pengembangan, debugging, dan integrasi antar milestone.

Secara umum, sistem dibagi menjadi beberapa modul utama sebagai berikut:

Modul	Tanggung Jawab
Lexer	Mengubah source code menjadi token
Parser	Membentuk parse tree dan AST
AST Builder	Mengubah parse tree menjadi AST
Symbol Table	Menyimpan metadata identifier
Semantic Analyzer	Melakukan semantic checking dan menghasilkan Decorated AST
Intermediate Code Generator	Menghasilkan intermediate code
Interpreter	Mengeksekusi intermediate code
Runtime Memory	Mengelola stack dan activation record

Hubungan antar modul dapat digambarkan sebagai berikut:



Arsitektur tersebut dipilih karena memungkinkan setiap tahapan compiler bekerja secara independen namun tetap terintegrasi dalam satu pipeline eksekusi.

3. Struktur Program

Implementasi Milestone 4 melibatkan beberapa file utama yang bertanggung jawab terhadap proses code generation dan runtime execution.

File	Fungsi
ast.hpp/.cpp	Representasi node-node AST
symbol_table.hpp/.cpp	Menyimpan informasi identifier program
semantic_analyzer.hpp/.cpp	Semantic checking dan Decorated AST
instruction.hpp	Representasi instruksi intermediate code
code_generator.hpp/.cpp	Intermediate code generation
interpreter.hpp/.cpp	Stack machine interpreter
main.cpp	Entry point compiler

Struktur tersebut dipilih agar proses semantic analysis, code generation, dan execution dapat dipisahkan menjadi modul-modul independen sehingga maintainability program menjadi lebih baik.

Selain itu, pemisahan modul juga membantu proses debugging karena setiap komponen dapat diuji secara terpisah.

4. Abstract Syntax Tree dan Semantic Analysis

Pada milestone sebelumnya, compiler telah menghasilkan Decorated AST melalui semantic analysis. Oleh karena itu, AST menjadi struktur utama yang digunakan pada proses intermediate code generation.

Semantic analyzer melakukan traversal terhadap AST menggunakan pendekatan Depth-First Search (DFS). Selama traversal berlangsung, semantic analyzer melakukan:

- type checking,
- scope resolution,
- identifier validation,
- dan symbol lookup.

Selain itu, semantic analyzer juga mendekorasi node AST dengan informasi semantic tambahan seperti:

- TypeCode,
- tab_index,
- lexical level,
- dan scope reference.

Sebagai contoh, node identifier akan menyimpan:

- tipe data identifier,
- posisi identifier pada symbol table,
- serta lexical level tempat identifier dideklarasikan.

Informasi tersebut digunakan oleh code generator untuk menentukan:

- jenis instruksi yang dihasilkan,
- memory address variable,
- dan lexical scope access selama runtime.

5. Symbol Table dan Scope Management

Compiler Arion menggunakan symbol table untuk menyimpan metadata seluruh identifier pada program. Implementasi symbol table dibagi menjadi beberapa tabel utama, yaitu TAB, BTAB, ATAB.

TAB digunakan untuk menyimpan identifier, seperti:

- variabel,
- konstanta,
- tipe data,
- procedure,
- dan function.

Setiap entri pada TAB menyimpan informasi seperti:

- nama identifier,
- lexical level,
- tipe data,
- kategori object,
- memory address,
- dan reference metadata.

Sedangkan BTAB digunakan untuk menyimpan informasi block dan scope, sementara ATAB digunakan untuk menyimpan metadata array.

Untuk menangani nested scope, compiler menggunakan:

- lexical level,
- hierarchical scope,
- dan static link traversal.

Scope management dilakukan menggunakan:

- pushScope()
- popScope()
- lookup()
- dan enter()

Pendekatan tersebut memungkinkan compiler mendukung:

- nested procedure,
- nested function,
- recursive call,
- serta lexical scoping.

6. Representasi Intermediate Code

Intermediate code pada compiler Arion direpresentasikan dalam bentuk instruction list berbasis stack machine. Setiap instruksi terdiri atas opcode, lexical level, dan operand/value. Berdasarkan spesifikasi milestone, instruksi utama yang digunakan meliputi:

- LIT
- LOD
- STO
- CAL
- INT

- JMP
- JPC
- OPR
- RET

Sedangkan operasi-operasi aritmatika maupun logika direpresentasikan menggunakan instruksi OPR dengan operation number tertentu seperti:

- ADD
- SUB
- MUL
- DIV
- MOD
- EQL
- NEQ
- GTR

Sebagai contoh, statement:

```
x := 10;
```

diterjemahkan menjadi:

```
LIT 0 10
STO 0 3
```

Instruksi LIT digunakan untuk memasukkan literal ke stack, sedangkan STO digunakan untuk menyimpan value ke alamat tertentu pada memory.

Representasi berbasis stack machine dipilih karena lebih sederhana, tidak memerlukan register allocation, dan mempermudah translasi expression dari AST.

7. Intermediate Code Generator

Intermediate code generator bertugas menerjemahkan Decorated AST menjadi instruction list executable.

Generator menggunakan pendekatan recursive Depth-First Traversal terhadap AST. Setiap jenis node AST diterjemahkan menjadi kombinasi instruksi tertentu.

Sebagai contoh:

- AssignNode menghasilkan instruksi assignment.
- BinaryExprNode menghasilkan operasi aritmatika.
- IfNode menghasilkan conditional jump.
- WhileNode menghasilkan loop translation.
- ProcedureCallNode menghasilkan procedure call.

Pada binary expression, generator akan melakukan traversal terhadap operand kiri dan operand kanan terlebih dahulu sebelum menghasilkan instruksi operasi. Pendekatan ini

menyerupai post-order traversal sehingga operand telah tersedia pada stack ketika operasi dijalankan oleh interpreter.

Sebagai contoh, expression:

```
a + b
```

diterjemahkan menjadi:

```
LOD 0 a
LOD 0 b
OPR 0 ADD
```

Generator juga memanfaatkan informasi semantic, seperti TypeCode, tab_index, lexical level, dan symbol table reference untuk menentukan address variable, lexical scope access, serta jenis operasi yang valid.

Selain expression dan assignment, generator juga mendukung array access, relational expression, procedure/function call, nested scope, dan runtime control flow.

8. Translasi Control Flow

Salah satu bagian penting pada code generation adalah translasi control flow statement menjadi instruksi jump.

Karena interpreter bekerja secara sequential, maka struktur seperti IF, WHILE, FOR, REPEAT, dan CASE perlu diterjemahkan menjadi label, unconditional jump, serta conditional jump.

Statement IF diterjemahkan menggunakan:

- 1) Evaluasi kondisi
- 2) Conditional jump (JPC)
- 3) Eksekusi body statement
- 4) Jump ke akhir statement

Sedangkan loop diterjemahkan menggunakan:

- 1) Label awal loop
- 2) Evaluasi kondisi
- 3) Conditional jump keluar loop
- 4) Eksekusi body
- 5) Jump kembali ke awal loop

Sebagai contoh sederhana:

```
while x < 5 do
  x := x + 1;
```

akan diterjemahkan menjadi kombinasi:

- label loop,

- evaluasi kondisi,
- JPC,
- body statement,
- dan JMP.

Pendekatan tersebut memungkinkan struktur program yang bersifat hierarkis diubah menjadi alur instruksi linear yang dapat dieksekusi oleh interpreter.

9. Interpreter dan Stack Machine

Interpreter pada compiler Arion dirancang sebagai virtual machine berbasis stack machine. Interpreter bertugas membaca instruction list, mengelola runtime memory, dan menjalankan instruksi secara sequential.

State utama interpreter meliputi:

- pc (program counter),
- sp (stack pointer),
- bp (base pointer),
- instruction list,
- dan stack memory.

Interpreter bekerja menggunakan siklus: 1) Fetch instruction, 2) Decode opcode, 3) Execute instruction

Instruksi dijalankan berdasarkan opcode yang dimiliki.

Sebagai contoh:

- LIT memasukkan literal ke stack
- LOD mengambil value dari memory
- STO menyimpan value ke memory
- JMP melakukan lompatan tanpa kondisi
- JPC melakukan lompatan berdasarkan kondisi
- CAL membuat activation record baru
- RET mengembalikan kontrol ke caller

Pendekatan stack machine dipilih karena lebih sederhana dan lebih mudah diintegrasikan dengan hasil translasi AST dibandingkan register-based machine.

10. Activation Record dan Runtime Memory

Untuk mendukung procedure maupun function call, interpreter menggunakan activation record atau stack frame.

Setiap stack frame menyimpan:

- static link,
- dynamic link,
- return address,
- local variables,
- parameter,
- dan temporary value.

Ketika procedure dipanggil menggunakan instruksi CAL, interpreter akan:

- 1) Membuat activation record baru.
- 2) Menyimpan static link.

- 3) Menyimpan dynamic link.
- 4) Menyimpan return address.
- 5) Memindahkan base pointer ke frame baru.

Sedangkan instruksi RET digunakan untuk menghapus frame aktif, mengembalikan base pointer, dan mengembalikan program counter ke caller. Pendekatan tersebut memungkinkan interpreter mendukung recursive call, nested scope, lexical scoping, dan hierarchical memory access. Selain itu, interpreter juga menggunakan static link traversal untuk mengakses variable pada outer scope berdasarkan lexical level.

11. Runtime Error Handling

Interpreter dilengkapi dengan beberapa mekanisme runtime validation untuk menjaga stabilitas eksekusi program.

Beberapa runtime error yang ditangani antara lain:

- division by zero,
- stack overflow,
- stack underflow,
- invalid jump target,
- corrupted stack frame,
- dan array out-of-bounds.

Sebelum menjalankan operasi tertentu, interpreter akan melakukan validasi kondisi runtime terlebih dahulu.

Sebagai contoh:

- ➔ operasi pembagian memeriksa divisor tidak bernilai nol,
- ➔ jump instruction memeriksa target address valid,
- ➔ dan array access memeriksa indeks masih berada dalam batas array.

Pendekatan tersebut membantu interpreter menghasilkan error message yang lebih aman dan informatif selama proses runtime execution berlangsung.

12. Hasil Implementasi

Hasil implementasi Milestone 4 memungkinkan compiler Arion:

- menghasilkan intermediate code dari Decorated AST,
- menerjemahkan control flow menjadi jump instruction,
- menjalankan program menggunakan interpreter,
- mendukung procedure/function call,
- mendukung nested scope,
- serta melakukan runtime execution menggunakan stack machine.

Selain itu, compiler juga telah mendukung:

- assignment statement,
- arithmetic expression,
- relational operation,
- conditional statement,
- loop,
- procedure/function call,
- dan array access.

Dengan implementasi tersebut, compiler Arion telah mampu menjalankan source code secara end-to-end mulai dari parsing hingga runtime execution.

Bab 3. Pengujian

1. Kasus 1

Input:

```
program HelloWorld;  
var  
  a, b: integer;  
begin  
  a := 5;  
  b := a + 10;  
  writeln(b);  
end.
```

Output (ini isi dari file output untuk milestone 4, sementara pada terminal ia akan menampilkan seluruh hasil konversi dari token hingga intermediate code)

--- INTERMEDIATE CODE ---

IDX	OP	LEVEL	VAL
0	JMP	0	1
1	INT	0	5
2	LIT	0	5
3	STO	0	3
4	LOD	0	3
5	LIT	0	10
6	OPR	0	2
7	STO	0	4
8	LOD	0	4
9	OPR	0	14
10	OPR	0	15
11	OPR	0	0

--- PROGRAM OUTPUT ---

15

2. Kasus 2

Input:

```
program LoopTest;  
var  
  i, sum: integer;  
begin  
  sum := 0;  
  i := 1;  
  while i <= 5 do begin  
    sum := sum + i;
```

```

    i := i + 1;
end;
writeln(sum);
end.

```

Output (ini isi dari file output untuk milestone 4, sementara pada terminal ia akan menampilkan seluruh hasil konversi dari token hingga intermediate code)

--- INTERMEDIATE CODE ---

IDX	OP	LEVEL	VAL
0	JMP	0	1
1	INT	0	5
2	LIT	0	0
3	STO	0	4
4	LIT	0	1
5	STO	0	3
6	LOD	0	3
7	LIT	0	5
8	OPR	0	12
9	JPC	0	19
10	LOD	0	4
11	LOD	0	3
12	OPR	0	2
13	STO	0	4
14	LOD	0	3
15	LIT	0	1
16	OPR	0	2
17	STO	0	3
18	JMP	0	6
19	LOD	0	4
20	OPR	0	14
21	OPR	0	15
22	OPR	0	0

--- PROGRAM OUTPUT ---

15

3. Kasus 3

Input:

```

program IfElseTest;
var
    x, y: integer;
begin

```

```

x := 10;
if x > 5 then begin
  y := 1;
end else begin
  y := 0;
end;
writeln(y);
end.

```

Output (ini isi dari file output untuk milestone 4, sementara pada terminal ia akan menampilkan seluruh hasil konversi dari token hingga intermediate code)

--- INTERMEDIATE CODE ---

IDX	OP	LEVEL	VAL
0	JMP	0	1
1	INT	0	5
2	LIT	0	10
3	STO	0	3
4	LOD	0	3
5	LIT	0	5
6	OPR	0	11
7	JPC	0	11
8	LIT	0	1
9	STO	0	4
10	JMP	0	13
11	LIT	0	0
12	STO	0	4
13	LOD	0	4
14	OPR	0	14
15	OPR	0	15
16	OPR	0	0

--- PROGRAM OUTPUT ---

1

4. Kasus 4

Input:

```

program ForLoopTest;
var
  i, result: integer;
begin
  result := 1;
  for i := 1 to 5 do begin

```

```

    result := result * i;
end;
writeln(result);
end.

```

Output (ini isi dari file output untuk milestone 4, sementara pada terminal ia akan menampilkan seluruh hasil konversi dari token hingga intermediate code)

--- INTERMEDIATE CODE ---

IDX	OP	LEVEL	VAL
-----	----	-------	-----

0	JMP	0	1
1	INT	0	5
2	LIT	0	1
3	STO	0	4
4	LIT	0	1
5	STO	0	3
6	LOD	0	3
7	LIT	0	5
8	OPR	0	12
9	JPC	0	19
10	LOD	0	4
11	LOD	0	3
12	OPR	0	4
13	STO	0	4
14	LOD	0	3
15	LIT	0	1
16	OPR	0	2
17	STO	0	3
18	JMP	0	6
19	LOD	0	4
20	OPR	0	14
21	OPR	0	15
22	OPR	0	0

--- PROGRAM OUTPUT ---

120

5. Kasus 5

Input:

```

program ProcedureTest;
var
  n, d: integer;

```

```

procedure printDouble(x: integer);
begin
  d := x * 2;
  writeln(d);
end;

begin
  n := 7;
  printDouble(n);
end.

```

Output (ini isi dari file output untuk milestone 4, sementara pada terminal ia akan menampilkan seluruh hasil konversi dari token hingga intermediate code)

```

--- INTERMEDIATE CODE ---
IDX  OP   LEVEL  VAL
0   JMP   0     10
1   INT   0      4
2   LOD   0      3
3   LIT   0      2
4   OPR   0      4
5   STO   1      4
6   LOD   1      4
7   OPR   0     14
8   OPR   0     15
9   RET   0      0
10  INT   0      5
11  LIT   0      7
12  STO   0      3
13  LOD   0      3
14  CAL   0      1
15  OPR   0      0

--- PROGRAM OUTPUT ---
14

```

6. Kasus 6

Input:

```

program ErrorDivZero;
var
  a, b: integer;

```

```

begin
  a := 10;
  b := 0;
  a := a div b;
  writeln(a);
end.

```

Output (ini isi dari file output untuk milestone 4, sementara pada terminal ia akan menampilkan seluruh hasil konversi dari token hingga intermediate code)

--- INTERMEDIATE CODE ---

IDX	OP	LEVEL	VAL
0	JMP	0	1
1	INT	0	5
2	LIT	0	10
3	STO	0	3
4	LIT	0	0
5	STO	0	4
6	LOD	0	3
7	LOD	0	4
8	OPR	0	5
9	STO	0	3
10	LOD	0	3
11	OPR	0	14
12	OPR	0	15
13	OPR	0	0

--- PROGRAM OUTPUT ---

Runtime error: Division by zero

7. Kasus 7

Input:

```

program RepeatTest;
var
  x: integer;
begin
  x := 1;
  repeat
    x := x * 2;
  until x >= 16;
  writeln(x);
end.

```


Output (ini isi dari file output untuk milestone 4, sementara pada terminal ia akan menampilkan seluruh hasil konversi dari token hingga intermediate code)

--- INTERMEDIATE CODE ---

IDX	OP	LEVEL	VAL
0	JMP	0	1
1	INT	0	4
2	LIT	0	1
3	STO	0	3
4	LOD	0	3
5	LIT	0	2
6	OPR	0	4
7	STO	0	3
8	LOD	0	3
9	LIT	0	16
10	OPR	0	10
11	JPC	0	4
12	LOD	0	3
13	OPR	0	14
14	OPR	0	15
15	OPR	0	0

--- PROGRAM OUTPUT ---

16

8. Kasus 8

Input:

```
program StringTest;  
begin  
    writeln('Hello, Arion!');  
end.
```

Output (ini isi dari file output untuk milestone 4, sementara pada terminal ia akan menampilkan seluruh hasil konversi dari token hingga intermediate code)

--- INTERMEDIATE CODE ---

IDX	OP	LEVEL	VAL
0	JMP	0	1
1	INT	0	3
2	LIT	0	0
3	OPR	0	14
4	OPR	0	15
5	OPR	0	0

--- PROGRAM OUTPUT ---

Hello, Arion!

Bab 4. Kesimpulan dan Saran

1. Kesimpulan

Pada Milestone 4 ini, compiler Arion berhasil dikembangkan agar mampu menerjemahkan Decorated Abstract Syntax Tree (AST) menjadi intermediate code dan menjalankan instruksi tersebut menggunakan interpreter berbasis stack machine. Implementasi yang dilakukan melanjutkan milestone sebelumnya yang telah menghasilkan AST dan semantic analyzer, sehingga compiler kini telah mampu menjalankan source code secara end-to-end mulai dari parsing hingga runtime execution.

Intermediate code generator berhasil diimplementasikan menggunakan pendekatan recursive Depth-First Traversal terhadap Decorated AST. Generator mampu menerjemahkan berbagai konstruksi program menjadi instruction list linear berbasis stack machine, seperti:

- assignment statement,
- arithmetic expression,
- relational operation,
- conditional statement,
- loop,
- procedure/function call,
- serta array access.

Selain itu, translasi control flow berhasil dilakukan menggunakan kombinasi conditional jump dan unconditional jump sehingga struktur seperti IF, WHILE, FOR, REPEAT, dan CASE dapat direpresentasikan dalam bentuk instruction flow linear.

Interpreter juga berhasil diimplementasikan menggunakan pendekatan virtual machine berbasis stack machine. Interpreter mampu:

- membaca dan mengeksekusi instruction list,
- mengelola runtime memory,
- membentuk activation record,
- menangani nested scope,
- serta mendukung procedure dan function call menggunakan static link dan dynamic link.

Berdasarkan hasil pengujian yang telah dilakukan, compiler Arion mampu menghasilkan intermediate code yang sesuai serta menjalankan program dengan output yang benar untuk berbagai kasus pengujian. Pengujian juga menunjukkan bahwa interpreter mampu menangani runtime error seperti division by zero dan invalid memory access secara aman tanpa menyebabkan program crash.

Dengan implementasi tersebut, tujuan utama Milestone 4 berhasil dicapai, yaitu membangun sistem intermediate code generation dan runtime execution menggunakan stack-based virtual machine.

2. Hasil Uji

Berdasarkan pengujian yang telah dilakukan, implementasi compiler Arion berhasil menangani berbagai jenis konstruksi program, antara lain:

- assignment dan arithmetic expression,
- conditional statement,

- while loop,
- for loop,
- repeat-until loop,
- procedure call,
- string output,
- serta runtime error handling.

Hasil pengujian menunjukkan bahwa:

- ★ Intermediate code berhasil dihasilkan sesuai struktur program input.
- ★ Interpreter mampu mengeksekusi instruction list secara sequential.
- ★ Runtime memory dan activation record berjalan dengan benar selama procedure/function call.
- ★ Translasi control flow menghasilkan alur eksekusi yang sesuai.
- ★ Runtime validation mampu mendeteksi error seperti division by zero.

Selain itu, pengujian juga menunjukkan bahwa pendekatan stack machine berhasil mendukung nested scope dan procedure execution menggunakan static link traversal.

3. Saran

Meskipun implementasi compiler Arion telah berhasil memenuhi kebutuhan Milestone 4, masih terdapat beberapa pengembangan yang dapat dilakukan pada masa mendatang.

Beberapa pengembangan yang dapat dilakukan antara lain:

- menambahkan optimization pass pada intermediate code,
- mengembangkan register-based virtual machine,
- menambahkan debugging dan tracing instruction,
- memperbaiki representasi string dan memory management,
- serta mengembangkan runtime error handling yang lebih lengkap.

Selain itu, compiler juga dapat dikembangkan lebih lanjut agar mendukung:

- recursive function yang lebih kompleks,
- parameter passing yang lebih fleksibel,
- dynamic memory allocation,
- serta optimization terhadap instruction execution.

Pengembangan tersebut diharapkan dapat meningkatkan efisiensi, fleksibilitas, dan kemampuan compiler Arion pada milestone berikutnya.

Lampiran

Tautan Repository Github: <https://github.com/Beeziebloop/ZZZ-Tubes-IF2224-2026>

Pembagian Tugas:

NIM	Nama	Pembagian Tugas
13524125	Muhammad Rafi Akbar	Traversal Decorated AST, Generate instruksi: INT, LIT, STO, LOD, OPR, JMP,.... Handle IF-ELSE, WHILE, prosedur/fungsi Output daftar instruksi TAC ke console/file, integrasi
13524135	Varistha Devi	Integrasi,Laporan
13524138	Ahmad Rinofaros Muchtar	Implementasi Stack + frame management (static link, dynamic link, return address) Siklus fetch → decode → execute Eksekusi semua instruksi: INT, LIT, LOD,... Semua operasi OPR (NEG, ADD, SUB, MUL, DIV, MOD, EQL, NEQ,...)

Referensi

1. <https://www.geeksforgeeks.org/compiler-design/phases-of-a-compiler/>
2. <https://www.geeksforgeeks.org/compiler-design/grouping-of-phases-in-compiler-design/>
3. <https://www.geeksforgeeks.org/compiler-design/intermediate-code-generation-in-compiler-design/>
4. <https://www.geeksforgeeks.org/compiler-design/introduction-to-intermediate-representation/>
5. <https://www.geeksforgeeks.org/compiler-design/three-address-code-compiler/>
6. <https://www.geeksforgeeks.org/computer-organization-architecture/stack-machine-in-computer-organisation>
7. https://cs420.epfl.ch/archive/20/c/11_interp-vms.html
8. <https://www.geeksforgeeks.org/compiler-design/access-links-and-control-links/>

9. <https://www.geeksforgeeks.org/dsa/static-and-dynamic-scoping/>
10. <https://www.tutorialspoint.com/article/what-is-translation-of-control-statements-in-compiler-design>
11. <https://www.geeksforgeeks.org/compiler-design/error-handling-compiler-design/>
12. https://www.meegle.com/en_us/topics/compiler-design/error-handling-in-compilers
13. <https://www.cs.princeton.edu/courses/archive/spr06/cos320/notes/Stack-handout.pdf>
14. https://user.it.uu.se/~kostis/Teaching/KT2-05/Slides/lecture13-VM_Interpreters.pdf
15. <https://users.sussex.ac.uk/~mfb21/compilers/slides/7-handout.pdf>
16. <https://www.math.wustl.edu/~freiwald/Math1322/arrays.pdf>